星之继承者 (Stellar Heir) — 交互式视频/3D 引擎技术说明文档

项目代号: `inherit-the-stars-demo` **GitHub 仓库**: Honyhuang0102/inherit-the-stars-demo **文档版本**: v1.0（基于 commit `b9f254d`） **整理日期**: 2026-04-03

目录

1. 系统架构与技术栈
2. 项目目录结构
3. 环境配置与启动
4. 后端服务 server.js
5. 数据结构格式 data.json
6. 创作者编辑器端 editor.html
7. 玩家播放器端 index.html
8. API 接口清单
9. 前端状态管理
10. 节点类型与行为规范
11. 已知问题与注意事项
12. 后续开发建议

---

1. 系统架构与技术栈

本项目是一个**轻量级全栈 Web 应用**，没有使用任何前端框架（React/Vue/Angular）或构建工具（Webpack/Vite），所有前端逻辑均通过原生 HTML5、CSS3 和 Vanilla JavaScript（ES6+）实现。这一设计使项目无需构建步骤，可直接部署运行。

层级	技术选型	说明
后端运行时	Node.js	服务器端 JavaScript 运行环境
Web 框架	Express.js v5	HTTP 路由与中间件
文件上传	Multer v2	内存存储，直接落盘
数据持久化	本地 JSON 文件	`data.json` 作为状态机，无数据库
前端渲染	Vanilla HTML/CSS/JS	无框架，无构建步骤
3D 渲染	Google `<model-viewer>` v3.4	Web Component，加载 `.glb` 模型
AI 全景生成	Blockade Labs API	基于参考图 + Prompt 生成 360° Skybox
字体	Google Fonts (Inter)	编辑器端使用

架构示意

```
浏览器
  ├── /            → public/index.html   (玩家播放器)
  └── /editor      → public/editor.html  (创作者编辑器)
        │
        ↕ REST API (JSON / FormData)
        │
  server.js (Express, Port 3000)
        │
        ├── GET/POST /api/*     → 读写 data.json
        ├── POST /api/upload-*  → 写入 public/videos/, public/panoramas/
        └── POST /api/generate-3d → 调用 Blockade Labs API (异步轮询)
```

---

2. 项目目录结构

```
inherit-the-stars-demo/
├── server.js                          # 后端主程序，所有 API 路由定义
├── data.json                          # 核心状态机数据（节点图）
├── package.json                       # 依赖配置（express, multer）
├── .env                               # 环境变量（不提交 Git，需手动创建）
├── .env.example                       # 环境变量模板（注意：内容已过时，见第3节）
├── .gitignore                         # 忽略 node_modules, .env,
data.backup.*.json
├── setup.sh                           # 历史遗留的脚手架脚本（已过时，勿参考）
├── README.md                          # 历史遗留的旧版说明（已过时，以本文档为准）
└── public/                            # 静态资源目录（Express 直接对外服务）
    ├── index.html                     # 玩家播放器引擎（约 1560 行）
    ├── editor.html                    # 创作者可视化编辑器（约 5050 行）
    ├── videos/                        # 上传的视频文件存储目录
    │   └── covers/                    # 视频封面图存储目录
    ├── panoramas/                     # 上传/生成的 360° 全景图存储目录
    └── models/                        # 3D 模型文件存储目录（.glb 格式）
```

> ***注意:*** *`data.backup.*.json` 是每次调用 `POST /api/save-data` 时自动生成的时间戳备份文件，存放在项目根目录，已被 `.gitignore` 排除。*

---

3. 环境配置与启动

3.1 依赖安装

```
npm install
```

项目运行时依赖：

- `express` ^5.2.1
- `multer` ^2.1.1
- `tencentcloud-sdk-nodejs-ai3d` ^4.1.203（已安装但当前代码未使用，为历史遗留依赖）

3.2 环境变量配置

在项目根目录创建 `.env` 文件。**注意：`.env.example` 文件内容已过时**，其中只包含腾讯云密钥，但当前后端实际使用的是 Blockade Labs API。正确的 `.env` 内容如下：

```
# Blockade Labs API 密钥（用于 AI 全景图生成）
# 获取地址：https://skybox.blockadelabs.com/
BLOCKADE_LABS_API_KEY=your_api_key_here
```

若不配置此密钥，`POST /api/generate-3d` 接口将返回 500 错误，但其他功能（视频上传、全景图上传、数据保存）不受影响。

3.3 启动服务

```
node server.js
# 或
npm start
```

服务启动后：

- **玩家端**: http://localhost:3000/
- **编辑器端**: http://localhost:3000/editor

---

4. 后端服务 server.js

`server.js` 是整个项目的后端核心，约 502 行。主要职责包括：静态文件服务、游戏数据读写、媒体文件上传、AI 全景图生成任务管理。

4.1 关键常量与配置

常量	值	说明
`PORT`	`3000`	服务监听端口
`DATA_PATH`	`./data.json`	状态机数据文件路径
`PUBLIC_DIR`	`./public`	静态资源根目录
`PANORAMA_DIR`	`./public/panoramas`	全景图存储目录
`MODELS_DIR`	`./public/models`	3D 模型存储目录
`VIDEOS_DIR`	`./public/videos`	视频存储目录
`COVERS_DIR`	`./public/videos/covers`	封面图存储目录
`BLOCKADE_API_BASE`	`https://backend.blockadelabs.com/api/v1`	Blockade Labs API 地址
`SKYBOX_STYLE_ID`	`35`	M3 UHD Render 风格（高清科幻写实）
`INIT_STRENGTH`	`0.35`	参考图影响强度（0.11=最强，0.9=无影响）

4.2 内存状态

`jobStore`（`Map<jobId, jobObject>`）用于在内存中维护 AI 全景图生成任务的状态。**服务重启后，所有进行中的任务状态将丢失**。这是一个已知限制，后续可考虑持久化到 `data.json` 或 Redis。

4.3 原子写入机制

`POST /api/save-data` 采用以下安全写入流程，防止写入中断导致数据损坏：

1. 将当前 `data.json` 复制为 `data.backup.<timestamp>.json`（备份）。
2. 将新数据写入 `data.json.tmp`（临时文件）。
3. 使用 `fs.rename()` 将临时文件原子重命名为 `data.json`。

4. 若 `rename` 失败（跨设备等情况），降级为直接 `fs.writeFile()`。

---

5. 数据结构格式 data.json

`data.json` 是整个引擎的核心，定义了节点图的拓扑结构和所有媒体配置。

5.1 根结构

```
{
  "title": "星之继承者 · 交互演示",
  "start_node": "1",
  "nodes": {
    "1": {},
    "2": {}
  }
}
```

字段	类型	必填	说明
`title`	string	否	项目标题（仅展示用）
`start_node`	string	**是**	游戏启动时加载的第一个节点 ID
`nodes`	object	**是**	节点字典，键为节点 ID

5.2 节点对象通用字段

字段	类型	必填	说明
`id`	string	**是**	节点唯一标识符，当前为纯数字字符串（如 `"1"`）
`type`	string	**是**	节点类型，枚举值见 5.3
`title`	string	否	节点标题，在编辑器卡片和结局屏幕显示
`description`	string	否	节点描述，在结局屏幕显示
`poster_url`	string	否	封面/缩略图 URL，在编辑器卡片和视频加载前显示
`choices`	array	条件	选项数组，`branching` / `3d_model` / `panorama` 类型必须包含
`_pos`	object	否	编辑器画布坐标 `{"x": 170, "y": 182}`，仅供编辑器使用

5.3 节点类型专有字段

`branching` — 视频分支节点

字段	类型	说明
`video_url`	string	视频文件 URL（相对路径，如 `/videos/video_xxx.mp4`）
`branch_time`	number	分支触发时间（秒），与 `branch_at_second` 含义相同，两者并存以兼容旧数据
`branch_at_second`	number	同上，旧版字段名
`next_node`	string	直连跳转目标节点 ID（无 choices 时视频结束后自动跳转）
`orientation`	string	视频方向，`"portrait"` 或 `"landscape"`
`video_width`	number	视频宽度（像素）
`video_height`	number	视频高度（像素）
`_sliderMaxSec`	number	编辑器时间轴最大秒数（仅供编辑器使用）

`ending` — 结局节点

字段	类型	说明
`video_url`	string	结局视频 URL
`ending_tag`	string	结局标签文字（如 `"ENDING · SILENT VOID"`）
`description`	string	结局描述文本

`3d_model` — 3D 模型节点

字段	类型	说明
`model_url`	string	`.glb` 模型文件 URL
`camera_orbit`	string	初始相机视角（可选，如 `"0deg 75deg 105%"`）

`panorama` — 360° 全景图节点

字段	类型	说明
`panorama_url`	string	等距柱状投影全景图 URL（相对路径，如 `/panoramas/skybox_xxx.jpg`）
`poster_url`	string	缩略图 URL（通常为 Blockade Labs 返回的 `thumb_url`）

5.4 选项对象 Choice Object

存在于 `branching`、`3d_model`、`panorama` 节点的 `choices` 数组中。

```
{
  "label": "[ 悄悄收下一个神秘设备 ]",
  "sublabel": "",
  "target_node": "2",
  "hotkey": "1",
  "position": {
    "x": "49.7%",
    "y": "74.2%"
  },
  "timeWindow": [13.56, 15.07]
}
```

字段	类型	适用类型	说明
`label`	string	全部	热点按钮显示文字
`sublabel`	string	全部	副标题（当前 UI 未显示，预留字段）
`target_node`	string	全部	点击后跳转的目标节点 ID
`hotkey`	string	全部	键盘快捷键绑定（如 `"1"`，`"2"`）
`position.x`	string	`branching`，`panorama`	热点水平位置百分比（如 `"50%"`）
`position.y`	string	`branching`，`panorama`	热点垂直位置百分比（如 `"74%"`）
`position`	string	`3d_model`	3D 空间坐标字符串 `"x y z"`（如 `"0.5 1.2 -0.3"`）
`normal`	string	`3d_model`	法线向量字符串 `"nx ny nz"`（如 `"0 1 0"`）
`timeWindow`	array	`branching`	热点显示时间窗口 `[开始秒, 结束秒]`
`type`	string	`branching`	热点类型，`"polygon"` 为多边形热点，默认为文字按钮
`points`	array	`branching`	多边形热点顶点坐标数组（仅 `type: "polygon"` 使用）

5.5 完整数据示例


```json
{
  "title": "星之继承者 · 交互演示",
  "start_node": "1",
  "nodes": {
    "1": {
      "id": "1",
      "type": "branching",
      "title": "序章 · 初醒",
      "video_url": "/videos/video_1774968376212.mp4",
      "poster_url": "/videos/covers/video_1774968376212_cover.jpg",
      "orientation": "portrait",
      "video_width": 720,
      "video_height": 1280,
      "_sliderMaxSec": 30,
      "choices": [
        {
          "label": "[ 悄悄收下神秘设备 ]",
          "sublabel": "",
          "target_node": "2",
          "hotkey": "1",
          "position": { "x": "49.7%", "y": "74.2%" },
          "timeWindow": [13.56, 15.07]
        },
        {
          "label": "[ 拒绝，保持警惕 ]",
          "sublabel": "",
          "target_node": "3",
          "hotkey": "2",
          "position": { "x": "49.7%", "y": "83.5%" }
        }
      ],
      "_pos": { "x": 170, "y": 182 }
    },
    "2": {
      "id": "2",
      "type": "panorama",
      "title": "飞船控制室",
      "panorama_url": "/panoramas/skybox_12345.jpg",
      "poster_url": "https://cdn.blockadelabs.com/thumb_xxx.jpg",
      "choices": [
        {
          "label": "[ 启动引擎 ]",
          "target_node": "4",
```

```
          "position": { "x": "60%", "y": "50%" }
        }
      ],
      "_pos": { "x": 500, "y": 120 }
    },
    "3": {
      "id": "3",
      "type": "ending",
      "title": "沉默的虚空",
      "video_url": "/videos/ending_001.mp4",
      "ending_tag": "ENDING · SILENT VOID",
      "description": "你选择了沉默，星际的大门就此关闭。",
      "_pos": { "x": 500, "y": 340 }
    }
  }
}
```

---

6. 创作者编辑器端 editor.html

编辑器是一个约 5050 行的单文件应用，提供可视化节点图编辑界面。

6.1 整体布局

```
+-------------------------------------------------------------+
|  工具栏 (Toolbar)                                            |
+------+-------------------------------------+---------------+
|      |                                     |               |
| 侧边栏|          画布 (Canvas)              |  属性面板     |
|      |    节点卡片 + SVG 连线              |  (Inspector)  |
|      |                                     |               |
|      |                                     |  小预览屏     |
|      |                                     |  (Preview)    |
+------+-------------------------------------+---------------+
```

编辑器还包含两个覆盖层（Overlay）：

- **沉浸式编辑模式**（`#immersive-overlay`）：全屏覆盖，直接在媒体画面上编辑热点。
- **AI 3D 生成向导**（`#ai3d-modal-overlay`）：弹出式 Modal，引导用户生成全景图。

6.2 前端状态对象 state

编辑器的所有运行时状态集中在全局 `state` 对象中：

```
const state = {
  gameData: null,           // 完整的 data.json 数据对象
  nodes: {},                // nodeId → { x, y, el, data }（画布上的节点实例）
  connections: [],          // [{ from, to, choiceIdx, label }]（连线数组）
  viewport: { x: 0, y: 0, scale: 1 }, // 画布视口变换
  panning: null,            // 画布平移状态
  dragging: null,           // 节点拖拽状态
  connecting: null,         // 连线绘制状态
  selBox: null,             // 框选状态
  selectedNodes: new Set(),// 当前选中的节点 ID 集合
  selectedNode: null,       // 当前单选节点 ID
  currentTool: 'select',    // 当前工具：'select' | 'connect'
  nodeIndexMap: {},         // nodeId → 序号（1-based，用于 UI 显示）
};
```

`nodeIndexMap` 是一个重要的辅助映射，在每次 `renderGraph()` 时重建，将节点的实际 key（可能是 `"1"` 或旧格式 `"node_xxx"`）映射到从 1 开始的显示序号。编辑器 UI 中所有对节点的引用均优先显示序号。

6.3 节点 ID 生成规则

`nextNodeId()` 函数负责生成新节点的 ID：

```
function nextNodeId() {
  let max = 0;
  Object.keys(state.gameData.nodes || {}).forEach(k => {
    const nums = k.match(/\d+/g);
    if (nums) nums.forEach(ns => {
      const n = parseInt(ns, 10);
      if (!isNaN(n) && n > max) max = n;
    });
  });
  return String(max + 1);
}
```

该函数会扫描所有节点 key 中的数字（兼容纯数字格式 `"1"` 和旧格式 `"node_1774981622405"`），取最大值 +1，确保新节点 ID 全局唯一且单调递增。

6.4 Inspector 属性面板

Inspector 在用户单击或双击节点时打开，在右侧面板中显示。`openInspector(nodeId)` 函数根据节点类型动态生成 HTML 表单。

各类型 Inspector 字段对照：

字段	branching	ending	3d_model	panorama
节点标题	✓	✓	✓	✓
视频 URL / 上传区	✓	✓	—	—
全景图 URL	—	—	✓	✓
分支时间	✓	—	—	—
选项列表	✓	—	✓	✓
结局标签	—	✓	—	—
结局描述	—	✓	—	—

`applyInspector(nodeId)` 函数负责将表单数据写回 `state.gameData.nodes[nodeId]`。其中对 `target_node` 字段有特殊处理：若用户输入的是纯数字，会先尝试通过 `nodeIndexMap` 反向查找实际节点 key，找不到则直接使用输入值（兼容节点 key 本身就是数字的情况）。

6.5 沉浸式编辑模式

通过工具栏按钮或节点卡片操作进入。核心函数 `loadImmersiveNode(nodeId)` 根据节点类型分流：

节点类型	沉浸模式行为
`branching` / `ending`	在全屏画面中播放视频，可拖拽热点按钮定位
`3d_model`	显示 `<model-viewer>`，可在 3D 空间中打点（3D 打点模式）
`panorama`	显示全景图预览（静态图片），可拖拽热点按钮定位

沉浸模式下的热点拖拽通过 `initImmersiveDrag()` 实现，拖拽结束后实时更新 `choice.position.x` 和 `choice.position.y`。

6.6 AI 全景图生成向导

通过右键菜单「添加 3D 场景（AI）」触发，核心函数 `startAI3DGeneration()`：

1. 截取当前选中节点视频的最后一帧（通过 `<canvas>` 绘制 `<video>` 帧）。
2. 将尾帧图片和用户输入的 Prompt 提交到 `POST /api/generate-3d`，获取 `jobId`。
3. 每 5 秒轮询 `GET /api/generate-3d/status/:jobId`，更新进度条。
4. 生成成功后，自动创建新的 `panorama` 类型节点并写入 `state.gameData`，调用 `saveData()` 保存。

6.7 自动保存

编辑器在 `DOMContentLoaded` 后启动一个 30 秒间隔的定时器，自动将当前节点坐标（`_pos`）写回 `state.gameData`，并调用 `POST /api/save-data` 静默保存。保存成功后更新工具栏中的「自动保存 HH:MM:SS」时间戳，不弹出 Toast 提示。

---

7. 玩家播放器端 index.html

播放器是约 1560 行的单文件应用，负责解析 `data.json` 并提供交互式播放体验。

7.1 整体 UI 结构

播放器在桌面端以「手机壳」（`#phone-shell`，390×844px）形式呈现，内部包含：

```
#phone-shell
  ├── #loading-screen       加载动画（约 1.4 秒后自动消失）
  ├── #error-screen         错误提示（无法连接后端时显示）
  ├── #tap-start-overlay    移动端点击解锁遮罩（绕过自动播放限制）
  ├── #hud-header           HUD 顶部（标题 + 系统状态）
  ├── #video-container      视频元素动态注入区
  ├── #hotspot-layer        热点按钮层（绝对覆盖）
  │   └── #hotspot-svg      SVG 多边形热点层
  ├── #model3d-container    3D 模型容器（model-viewer）
  ├── #ending-screen        结局展示屏
  ├── #hud-footer           HUD 底部（节点标签 + 时间）
  └── [动态] #pano-container  全景图容器（loadPanoramaNode 创建）
```

7.2 引擎状态对象 engine

```
const engine = {
  gameData:      null,     // 完整 JSON 数据
  currentNodeId: null,     // 当前活跃节点 ID
  activeVideo:   null,     // 当前活跃 <video> 元素引用
  choiceShown:   false,    // 是否已弹出热点选项（防重入）
  branchHandled: false,    // 是否已触发分支（防重入）
  transitioning: false,    // 是否正在切换节点（防重复点击）
  videoEnded:    false,    // 视频是否已结束（控制热点显隐逻辑）
  userInteracted: false,   // 用户是否已交互（解锁音频）
};
```

7.3 Video Pool 引擎

`videoPool`（`Map<nodeId, HTMLVideoElement>`）是播放器的核心缓存机制：

- **活跃视频**（`class="vid-active"`）：当前播放的视频，非静音，可见，`z-index: 1`。
- **预加载视频**（`class="vid-preload"`）：后台静默缓冲，静音，隐藏，`z-index: 0`。

当用户选择分支时，`handleChoice()` 直接调用 `loadNode(targetId)`，如果目标视频已在 Pool 中，则无缝切换（无黑屏）；否则冷启动加载。`releaseStaleVideos(keepIds)` 函数在每次节点加载后清理不在保留集合中的旧视频，防止内存泄漏。

7.4 节点加载分流逻辑

`loadNode(nodeId)` 是播放器的核心入口，根据节点类型分流：

```
loadNode(nodeId)
  ├── type === '3d_model'  → exitPanoramaMode() → load3DModelNode()
  ├── type === 'panorama'  → exit3DModelMode() → loadPanoramaNode()
  └── 其他（branching/ending）
        → exit3DModelMode() → exitPanoramaMode()
        → 从 videoPool 获取或创建 <video>
        → activateVideo() → 绑定事件监听
        → preloadNode() 预加载分支目标
```

7.5 热点系统

`showChoices(node)` 在视频结束（`onEnded` 事件）后被调用，动态创建热点按钮：

- **文字按钮热点**：`<button class="hotspot-btn">` 绝对定位于 `#hotspot-layer`，位置由 `choice.position.x/y` 决定。
- **多边形热点**：`<polygon>` 渲染到 `#hotspot-svg`，使用 `viewBox="0 0 100 100"` 百分比坐标系。

热点按钮支持 `is-ready` 状态（目标视频已预加载完成时显示绿色指示点）。

7.6 全景图引擎

`loadPanoramaNode(node)` 函数：

1. 在 `#phone-shell` 内动态创建 `#pano-container` 容器（首次调用时创建，复用时清空内容）。
2. 创建一个 200% 宽度的 `<div>` 包裹全景图 `<img>`，通过 `translateX()` 实现水平循环滚动效果。
3. 监听 `mousedown` / `touchstart` / `mousemove` / `touchmove` / `mouseup` / `touchend` 事件，实现拖拽旋转交互。
4. 调用 `renderPanoChoices(node)` 在全景图上渲染热点按钮。

7.7 3D 模型引擎

`load3DModelNode(node)` 函数：

1. 动态创建 `<model-viewer>` 元素，设置 `src`、`camera-controls`、`auto-rotate` 等属性。
2. 遍历 `node.choices`，为每个 choice 创建 `<button slot="hotspot-N">` 并设置 `data-position` 和 `data-normal` 属性，由 `<model-viewer>` 负责 3D 空间定位。
3. 点击热点后调用 `handleChoice(choice)` 跳转。

7.8 移动端兼容

- 所有 `<video>` 元素设置 `playsinline` 和 `webkit-playsinline` 属性，确保 iOS Safari 内联播放。
- 初始以 `muted` 状态播放，绕过浏览器自动播放策略限制。
- 若 `play()` 被浏览器拦截，显示 `#tap-start-overlay`，等待用户点击后解锁音频并重试播放。

---

8. API 接口清单

8.1 页面路由

方法	路径	说明
`GET`	`/`	玩家播放器（`public/index.html`，由静态服务提供）
`GET`	`/editor`	创作者编辑器（`public/editor.html`）

8.2 游戏数据 API

`GET /api/game-data`

读取并返回完整的 `data.json` 内容。

- **调用方**: 播放器端初始化、编辑器端初始化

- **响应**: `200 OK`，返回完整的游戏数据 JSON 对象
- **错误**: `500`，无法读取或解析 `data.json`

`POST /api/save-data`

保存完整的游戏状态机数据。

- **调用方**: 编辑器端（手动保存 / 30 秒自动保存）
- **请求体**: `Content-Type: application/json`

```
{
  "title": "...",
  "start_node": "1",
  "nodes": {}
}
```

- **校验规则**: `nodes` 字段必须存在且为对象；`start_node` 字段必须存在且为字符串；`nodes[start_node]` 必须存在。
- **响应**: `200 OK`

```
{ "ok": true, "message": "保存成功", "timestamp": "2026-04-03T..." }
```

- **错误**: `400` 校验失败，`500` 文件写入失败

8.3 媒体上传 API

`POST /api/upload-video`

上传视频文件，可选同时上传封面图。

- **请求体**: `Content-Type: multipart/form-data`

字段名	类型	必填	说明
`video`	File	**是**	视频文件（mp4/webm/mov/avi/mkv/mpeg/ogg）
`cover`	string	否	封面图 Base64 字符串（含 `` 前缀）
`width`	number	否	视频宽度（像素）
`height`	number	否	视频高度（像素）

- **文件大小限制**: 500 MB（视频），10 MB（封面）
- **响应**: `200 OK`

```
{
  "ok": true,
  "videoUrl": "/videos/video_1774968376212.mp4",
  "coverUrl": "/videos/covers/video_1774968376212_cover.jpg",
  "width": 720,
  "height": 1280,
  "orientation": "portrait",
  "message": "视频上传成功"
}
```

`POST /api/upload-panorama`

上传 360° 等距柱状投影全景图。

- **请求体**: `Content-Type: multipart/form-data`

字段名	类型	必填	说明
`image`	File	**是**	全景图文件（jpg/png/webp）

- **文件大小限制**: 20 MB
- **响应**: `200 OK`

```
{
  "ok": true,
  "url": "/panoramas/pano_1774968376212.jpg",
  "message": "全景图上传成功"
}
```

8.4 AI 全景图生成 API

`POST /api/generate-3d`

提交 AI 全景图生成任务（异步，立即返回 `jobId`）。

- **前置条件**: 必须在 `.env` 中配置 `BLOCKADE_LABS_API_KEY`
- **请求体**: `Content-Type: multipart/form-data`

字段名	类型	必填	说明
`image`	File	**是**	视频尾帧参考图（JPEG）
`prompt`	string	**是**	场景描述文字（英文效果更佳）

- **响应**: `200 OK`

```
{
  "ok": true,
  "jobId": "skybox_12345_1774968376212",
  "message": "全景图生成任务已提交，请轮询状态"
}
```

- **后台行为**: 服务器每 5 秒轮询 Blockade Labs API，最多轮询 300 次（约 25 分钟）。生成完成后自动下载全景图到 `public/panoramas/` 目录。

`GET /api/generate-3d/status/:jobId`

查询 AI 全景图生成任务状态。

- **路径参数**: `jobId` （由 `POST /api/generate-3d` 返回）

- **响应**: `200 OK`

`status` 值	说明
`pending`	任务已提交，等待处理
`processing`	正在生成中
`done`	生成完成
`error`	生成失败

```
{
  "ok": true,
  "status": "done",
  "panoramaUrl": "/panoramas/skybox_12345.jpg",
  "thumbUrl": "https://cdn.blockadelabs.com/...",
  "error": null
}
```

- **错误**: `404`，任务不存在或服务重启后状态丢失

8.5 健康检查 API

`GET /api/health`

- **响应**: `200 OK`

```
{
  "status": "ok",
  "timestamp": "2026-04-03T...",
  "blockadeLabs": true,
  "skyboxStyleId": 35
}
```

---

9. 前端状态管理

9.1 编辑器端关键函数

函数	说明
`loadData()`	从 `/api/game-data` 加载数据，调用 `renderGraph()`
`renderGraph()`	清空画布，重建 `nodeIndexMap`，创建所有节点卡片和连线
`createNodeCard(nodeData, x, y)`	在画布上创建节点 DOM 元素，注册拖拽和点击事件
`addNode(type)`	创建新节点，调用 `nextNodeId()` 生成 ID
`deleteNode(id)`	删除节点及其相关连线，清理 `target_node` 引用
`openInspector(nodeId)`	打开属性面板，动态生成表单 HTML
`applyInspector(nodeId)`	将表单数据写回 `state.gameData`，刷新画布和预览
`saveData()`	将 `state.gameData` 提交到 `POST /api/save-data`
`updatePreviewPanel(nodeData)`	更新右侧小预览屏（视频/全景图/占位符）
`loadImmersiveNode(nodeId)`	进入沉浸式编辑模式，根据类型分流渲染
`startAI3DGeneration()`	启动 AI 全景图生成流程
`nextNodeId()`	生成全局唯一的递增节点 ID
`fitView()`	将画布视口适应所有节点（快捷键 F）
`showToast(msg, type)`	显示右上角 Toast 通知（`success` / `error` / `info`）

9.2 播放器端关键函数

函数	说明
`init()`	初始化：加载数据，显示 tap-start，加载起始节点
`loadNode(nodeId)`	节点加载主入口，根据类型分流
`load3DModelNode(node)`	加载 3D 模型节点
`loadPanoramaNode(node)`	加载全景图节点
`exit3DModelMode()`	退出 3D 模型模式，清理容器
`exitPanoramaMode()`	退出全景图模式，清理容器
`showChoices(node)`	渲染热点按钮（视频结束后调用）
`handleChoice(choice)`	处理用户选择，跳转到目标节点
`showEnding(node)`	显示结局屏幕
`preloadNode(nodeId)`	后台预加载视频（跳过 panorama/3d_model 类型）
`releaseStaleVideos(keepIds)`	释放不在保留集合中的旧视频
`renderPanoChoices(node)`	在全景图上渲染热点按钮

---

10. 节点类型与行为规范

10.1 节点类型颜色编码（编辑器）

类型	颜色	说明
起始节点（`start_node` 所指向的节点）	蓝色边框	通过 CSS class `is-start` 标记
`branching`	琥珀色/橙色	最常见的视频分支节点
`ending`	紫色	结局终止节点
`3d_model`	深色主题	3D 模型交互节点
`panorama`	深色主题	360° 全景图节点

10.2 播放器端节点行为流程

```
视频节点 (branching)
  → 播放视频
  → 视频结束 (onEnded)
  → showChoices() 显示热点
  → 用户点击热点
  → handleChoice() → loadNode(target_node)

结局节点 (ending)
  → 播放视频
  → 视频结束 (onEnded)
  → showEnding() 显示结局屏幕
  → 用户点击"重新开始"
  → 清空 videoPool → loadNode(start_node)

3D 模型节点 (3d_model)
  → 暂停/隐藏视频
  → 显示 model-viewer
  → 用户点击 3D 热点
  → handleChoice() → loadNode(target_node)

全景图节点 (panorama)
  → 暂停/隐藏视频
  → 显示全景图（可拖拽旋转）
  → 用户点击热点按钮
  → handleChoice() → loadNode(target_node)
```

---

11. 已知问题与注意事项

11.1 配置文件不一致

`.env.example` 中只包含腾讯云密钥（`TENCENT_SECRET_ID` / `TENCENT_SECRET_KEY`），而当前后端实际使用 `BLOCKADE_LABS_API_KEY`。腾讯云 SDK（`tencentcloud-sdk-nodejs-ai3d`）已安装但未被当前代码使用，为历史遗留依赖。后续开发者需注意：**以 `server.js` 中的实际代码为准，而非 `.env.example`**。

11.2 AI 任务状态内存存储

`jobStore` 使用 Node.js 内存 `Map` 存储任务状态，服务重启后所有进行中的任务状态将丢失，前端轮询会收到 404 错误。如需生产化，应将任务状态持久化到 `data.json` 或外部存储。

11.3 全景图预览精度

播放器端的全景图引擎使用 CSS `translateX()` 模拟等距柱状投影，是一种近似实现，不是真正的球面投影。在视角偏转较大时会有明显的图像拉伸变形。如需高精度 360° 查看，建议集成 Pannellum 或 A-Frame 库。

11.4 节点 ID 格式兼容

系统同时支持纯数字格式（`"1"`，`"2"`）和旧版 `node_xxx` 格式（`"node_1774981622405"`）的节点 ID。`nextNodeId()` 函数会扫描所有格式的 ID 并取最大数字 +1，确保不重复。建议新建节点统一使用纯数字格式。

11.5 视频上传大小限制

Express 中间件配置了 `600mb` 的 JSON 请求体限制，Multer 配置了 `500mb` 的视频文件限制。实际部署时可能受到 Nginx 等反向代理的 `client_max_body_size` 限制，需同步调整。

11.6 旧版文档与脚本

`README.md` 和 `setup.sh` 均为早期脚手架阶段的遗留文件，描述的是三节点 Demo 架构（`node_A/B/C`），与当前实现差异较大，不应作为参考。以本文档为准。

---

12. 后续开发建议

以下是基于当前架构，后续可以优先推进的功能方向：

优先级高：

全景图真实球面渲染方面，建议集成 Pannellum.js 替换当前的 CSS 近似方案，实现真正的 360° 球面投影和陀螺仪支持。任务状态持久化方面，将 `jobStore` 的任务状态写入

`data.json` 或 SQLite，解决服务重启后任务丢失问题。多用户/多项目支持方面，当前 `data.json` 是单文件单项目，可扩展为按项目 ID 存储多个 JSON 文件。

优先级中：

视频预加载策略优化方面，当前预加载所有分支目标，对于分支较多的节点会消耗大量带宽，可改为懒加载或优先级队列。节点图导出/导入方面，支持将 `data.json` 导出为标准格式，或从其他工具导入。键盘快捷键完善方面，编辑器中 Ctrl+Z 撤销、Ctrl+S 保存等快捷键尚未完整实现。

优先级低：

多边形热点编辑器方面，当前多边形热点（`type: "polygon"`）只能在 `data.json` 中手动配置，编辑器尚未提供可视化绘制工具。视频剪辑功能方面，在编辑器内支持视频裁剪和出入点设置，减少对外部工具的依赖。云端部署支持方面，添加 Docker 配置和 Nginx 反向代理配置，简化生产环境部署流程。

本文档由 Manus AI 根据源代码自动整理生成，版本对应 commit `b9f254d`。